

COMP9414 Assignment 1

Search and planning for an office delivery agent

Term 2, 2026

UNSW Sydney

Release	Sunday, 14 June 2026
Due	Friday, 3 July 2026, 5:00 PM AEST
Worth	20 marks
Submission	Electronic submission via Moodle

1 Introduction

This assignment explores how a goal-directed agent can solve route-finding and planning tasks in a small office environment. In week 1, we described search problems in terms of states, actions, goals, path costs, search trees, and frontiers. In week 2, we extended the same idea to planning problems, where states are sets of facts and actions are described by preconditions and effects.

You will implement and compare several search algorithms, then build a small STRIPS-style planner for an office delivery robot. The robot must reason about rooms, a parcel, a keycard, and a locked lab door. A route through the office is not enough: the robot must choose actions that are legal when attempted and that make the delivery goal true.

This assignment is focused on weeks 1 and 2, search and planning. You should not need to use later course topics.

2 Background

2.1 Agents and search

An agent perceives, decides, and acts in an environment. To solve a search problem, the agent needs a problem formulation:

- an initial state;
- a successor function or action model;
- a goal test;
- a path-cost function;
- a strategy for choosing the next frontier node.

Depth-first search (DFS), breadth-first search (BFS), uniform-cost search (UCS), and A* search differ mainly in the way they order the frontier. DFS follows a deep branch before backtracking. BFS explores by depth. UCS expands the cheapest path so far. A* combines path cost so far with a heuristic estimate of remaining cost.

2.2 Planning

Planning uses search, but it makes the structure of the world explicit. A classical planning problem can be written as

$$P = (O, s_0, g),$$

where O is a set of operators, s_0 is the initial state, and g is the goal condition. In STRIPS-style planning, each grounded action has:

- preconditions that must already hold;
- add effects that become true;
- delete effects that stop being true.

The transition rule is:

$$\text{Result}(s, a) = (s - \text{Del}(a)) \cup \text{Add}(a),$$

provided $\text{Pre}(a) \subseteq s$. Forward planning starts at the initial state, generates applicable actions, applies their effects, and searches until a generated state satisfies the goal.

3 Assignment specification

3.1 Provided examples and discussion check instances

The supplied examples are small JSON-style graph and planning problems. You must download the supplied data files from Moodle and use those files as the source of truth for your automated runs. The tables below document the file contents and the hand-trace graph, but they are not a replacement for loading the supplied files in your reusable evaluation cell. During the discussion session, your tutor may also ask you to run or reason about a small unseen instance using the same schema. This is a check that your solver implements the general algorithm rather than memorising one example; it is not a separate data split.

3.1.1 Distributed data files

The assignment data folder contains the files in Table 1. Place the `data/` folder next to your submitted notebook.

Table 1: Files supplied for the assignment.

File	Use
<code>data/graphs/office.json</code>	Part A hand-trace graph.
<code>data/graphs/graph1.json</code>	Small supplied weighted graph for search and A* comparison.
<code>data/graphs/graph2.json</code>	Larger supplied weighted graph for search and A* comparison.
<code>data/graphs/graph3.json</code>	Supplied weighted graph where BFS returns a shallow path that is not cheapest.
<code>data/planning/canonical_delivery.json</code>	Baseline STRIPS planning case.
<code>data/planning/unlocked_lab.json</code>	Planning case where the lab starts unlocked.

File	Use
<code>data/planning/lab_via_office_b.json</code>	Planning case where the lab door connects office B and the lab.
<code>data/manifest.json</code>	Provenance for all supplied files.

The sizes, seeds, and example names for the supplied files are recorded in `data/manifest.json`.

3.1.2 Graph problem schema

Each graph problem is an undirected graph:

```
{
  "nodes": [{"id": "corridor", "x": 0.0, "y": 0.0}, ...],
  "edges": [{"u": "corridor", "v": "mail_room", "cost": 2.0}, ...],
  "start": "corridor",
  "goal": "lab"
}
```

All edge costs are positive. Node coordinates are used only for the straight-line heuristic in Part C. Unless a question says otherwise, generate successors deterministically by neighbouring node id in alphabetical order. This is a successor-generation rule; frontier tie-breaking for UCS and A* is specified in Section 3.2. Stop when the goal is expanded, not when it is first generated. Graph files may include extra fields such as `id` or `metadata`; your solver may ignore them.

3.1.3 Planning problem schema

Each planning problem uses the office-delivery STRIPS schema from the week 2 tutorial. The supplied variants are:

- `canonical_delivery`;
- `unlocked_lab`;
- `lab_via_office_b`.

You may inspect these variants with:

```
ai9414 demo strips --example canonical_delivery
ai9414 demo strips --example unlocked_lab
ai9414 demo strips --example lab_via_office_b
```

If you use the installed `ai9414` package for inspection or helper functions, use version 0.1.4 or newer.

Planning files may include a `metadata` object; your solver may ignore it.

3.2 Stable solver interfaces

Your submitted notebook must define the following functions in executable code cells before the reusable evaluation cell.

```
def solve_graph(problem: dict, algorithm: str) -> dict:
    ...
```

The argument `algorithm` must accept exactly: `dfs`, `bfs`, `ucs`, and `astar`.

```
def solve_planning(problem: dict) -> dict:
    ...
```

Both functions must return dictionaries using the fields in Table 2.

Table 2: Required solver return fields.

Field	Meaning
<code>status</code>	<code>found</code> , <code>not_found</code> , or <code>error</code> .
<code>path</code>	For graph search, the returned node-id path from start to goal. Use an empty list if no path is found.
<code>plan</code>	For planning, the returned grounded action sequence. Use an empty list if no plan is found.
<code>cost</code>	For graph search, the total returned path cost. For planning, use the number of actions in the plan. Use <code>None</code> if no solution is found.
<code>expanded_order</code>	Ordered list of expanded graph nodes or canonical state identifiers.
<code>expanded_count</code>	Number of graph nodes or planning states expanded.
<code>generated_count</code>	Number of graph nodes or planning states generated, including repeated candidates if your algorithm considered them.
<code>frontier_peak</code>	Largest frontier size reached during the search.
<code>message</code>	Optional short diagnostic string for <code>error</code> results.

For deterministic marking, do not use random tie-breaking. DFS and BFS should use the successor-generation order from the graph schema above, with DFS pushing successors in reverse order if needed so that the alphabetically first successor is popped next. For UCS and A*, if two frontier entries have the same primary priority value, compare the complete path tuples lexicographically. If there is still a tie, keep the entry that was inserted first.

The live Python interface in `ai9414 demo strips` uses a smaller function named `solve_planner(problem)` whose required return fields are `algorithm`, `status`, and `plan`. That interface is useful for testing a plan in the browser, but the assignment function `solve_planning(problem)` must return all assignment fields in Table 2, including the search metrics.

3.3 Solution format

Your complete solution must be submitted as a Jupyter notebook named `zID_assignment1.ipynb`. The notebook must contain your algorithm implementations, written answers, tables, plots or screenshots, and executed outputs for the supplied examples.

The notebook must define `solve_graph(problem, algorithm)` and `solve_planning(problem)` as normal Python functions. These functions are the marking interface and must be available to later cells in the notebook.

The notebook must show the hand traces, comparison tables, plots or screenshots, written answers requested in Parts A–E, and the reusable evaluation cell described in Section 9.1.

4 Part A: problem formulation and search traces

4.1 Office graph formulation

Use the small weighted office graph in Table 3. Treat each room as a state and each edge as a reversible movement action.

From	To	Cost
corridor	mail_room	2
corridor	office_a	2
corridor	office_b	3
mail_room	storage	2
storage	lab	7
office_a	lab	8
office_b	lab	3
office_a	office_b	2

Table 3: Supplied small weighted office graph. Start at `corridor`; goal is `lab`.

1. State the initial state, goal test, actions, transition model, and path cost for this graph.
2. Draw or tabulate the first three frontier updates for DFS, BFS, and UCS.
3. Report the full expansion order for DFS, BFS, and UCS.
4. Report the returned path and returned cost for each algorithm.
5. Explain why BFS and UCS can return different paths on this graph.

4.2 Search tree versus state space

In your solution document, include a short explanation of the difference between a state space graph and a search tree. Use the office graph as the example, and mention why the same room may appear more than once in a search tree even though it is a single state in the state space.

5 Part B: implement graph search

5.1 Algorithms

Implement `solve_graph(problem, algorithm)` in `zID_search.py`. You must implement:

- DFS over paths, using a stack frontier;
- BFS over paths, using a queue frontier;
- UCS over paths, using cumulative path cost as priority;
- A* search, using $f(n) = g(n) + h(n)$, where $g(n)$ is path cost so far and $h(n)$ is straight-line distance to the goal.

Your graph search code must work for the graph in Part A and for other graphs with the same schema. You may use Python standard-library data structures such as `collections.deque` and `heapq`. You may not call a library search function that directly solves the problem for you.

5.2 Evaluation

Run all four algorithms on:

- `data/graphs/office.json`, the small graph from Table 3;
- `data/graphs/graph2.json`, the larger supplied graph;
- `data/graphs/graph3.json`, the supplied graph where BFS finds a shallow path that is not the cheapest path.

For each run, report the path, cost, expanded count, generated count, frontier peak, and expansion order. Present a comparison table and at least one plot comparing expanded count and returned cost across algorithms.

6 Part C: A* heuristic behaviour

6.1 Straight-line heuristic

For A* search, use the Euclidean distance between the current node coordinates and the goal coordinates:

$$h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}.$$

Use this function for every graph.

6.2 Empirical comparison

Choose at least two weighted graphs with node coordinates. One should be small enough to inspect by hand; the other should have enough branches to show differences in frontier growth. For each graph:

1. run UCS and A*;
2. compare returned cost, expanded count, generated count, and frontier peak;
3. identify one point in the trace where A* chooses a different frontier entry from UCS because of the heuristic term;
4. explain, in empirical terms, how the heuristic changed the search effort relative to UCS;
5. discuss one limitation of relying on coordinate distance as a guide in a graph whose edge costs need not equal geometric distance.

Your explanation should be based on frontier choices and observed metrics.

7 Part D: STRIPS modelling and forward planning

7.1 Office delivery domain

Use the office delivery domain from week 2. The core objects are: `robot`, `parcel`, `keycard`, `lab_door`, and the rooms `corridor`, `mail_room`, `office_a`, `office_b`, and `lab`. The goal is usually:

$$\{\text{at}(\text{parcel}, \text{lab})\}.$$

The relevant action schemas are:

- `move(robot, from, to);`
- `pickup_keycard(robot, keycard, room);`
- `pickup_parcel(robot, parcel, room);`
- `unlock_door(robot, keycard, lab_door, here, there);`
- `drop_parcel(robot, parcel, room).`

7.2 Plan diagnosis

Consider the following proposed plan for `lab_via_office_b`, where the locked lab door connects `office_b` and `lab`:

1. `move(robot, corridor, mail_room)`
2. `pickup_parcel(robot, parcel, mail_room)`
3. `move(robot, mail_room, corridor)`
4. `move(robot, corridor, office_b)`
5. `move(robot, office_b, lab)`
6. `drop_parcel(robot, parcel, lab)`

In your solution document:

1. identify the first action that is not applicable when reached;
2. name every missing precondition or false fact that blocks that action;
3. explain why simply inserting `unlock_door` immediately before the blocked action is still not enough;
4. give a valid repaired plan;
5. identify where your repaired plan first makes `unlocked(lab_door)` true;
6. explain how the final state satisfies `at(parcel, lab)`.

7.3 Forward BFS planner

Implement `solve_planning(problem)` in your notebook. Your planner must implement the forward BFS search loop. The suggested approach is to use the `ai9414.strips` helpers documented in Appendix A; alternatively, you may implement the STRIPS transition machinery yourself. In either case, your own code must:

- get or construct the initial fact set;
- get or enumerate grounded actions deterministically;
- get or compute successor states using STRIPS preconditions and effects;
- test whether a state satisfies all goal facts;
- avoid revisiting previously seen states;
- use a queue frontier for forward BFS;
- return the required fields from Table 2.

For state identifiers in `expanded_order`, use a stable canonical representation such as sorted tuples of facts. You may choose the exact representation, but keep it deterministic and readable during discussion.

7.4 Planning comparison

Run your forward planner on:

- `canonical_delivery`;
- `unlocked_lab`.

Create a comparison table with status, plan length, expanded count, generated count, frontier peak, and final plan. Explain why the unlocked case can be solved with fewer actions.

8 Evaluation metrics

Report the following metrics throughout the assignment:

- **Returned cost:** total edge cost for graph search; number of actions for planning.
- **Expanded count:** number of nodes or states removed from the frontier and expanded.
- **Generated count:** number of candidate successors produced.
- **Frontier peak:** maximum frontier size during the run.
- **Expansion order:** ordered list of expanded node ids or canonical planning state ids.

When comparing algorithms, do not compare only the returned path. A useful analysis should connect the returned cost to the search effort and the frontier strategy.

9 Assessment and evaluation

9.1 Evaluation process

Your tutor will evaluate your assignment during a scheduled discussion session. Discussions start in the week beginning Monday, 6 July 2026. Your tutor will arrange the timing, which may be during tutorial hours or directly with their students. You must attend your arranged discussion. If you do not discuss your submission, you will receive zero marks for the assignment.

Your notebook must include a reusable evaluation cell. It must:

1. load graph and planning JSON files using the assignment schemas;
2. call the `solve_graph` and `solve_planning` functions defined earlier in the notebook;
3. run all required algorithms;
4. print the returned fields in a readable table;
5. write the same results to `zID_results.json`.

Before submission, run this evaluation cell on the supplied examples. During the discussion, your tutor may ask you to run the same code on a small check instance or to trace the algorithm by hand on a modified instance. If you do not include the evaluation cell, you will lose 2 marks from your total assignment mark.

9.2 Assessment breakdown

Table 4: Assignment assessment breakdown.

Assessment component	Marks
Assignment implementation and notebook	
Part A: Problem formulation and hand search traces	1.5
Part B: Graph search implementation and comparison	2.0
Part C: Heuristic behaviour analysis	1.5
Part D: STRIPS modelling and forward BFS planning	3.0
Understanding and discussion	
Group 1: Foundations	4.0
Group 2: Search behaviour	4.0
Group 3: Planning models	4.0
Total	20.0

For the discussion component, your tutor will ask you about one topic from each group:

- **Foundations:** problem formulation; state space versus search tree; frontier and explored set; deterministic tie-breaking.
- **Search behaviour:** DFS versus BFS; UCS cost ordering; A* implementation and heuristic effects.
- **Planning models:** STRIPS preconditions and effects; forward BFS planning; plan diagnosis; planning comparison.

10 Submission requirements

Submit a single ZIP archive named `zID_assignment1.zip` via Moodle. It must contain:

1. `zID_assignment1.ipynb`, with output cells executed for the supplied examples;
2. `zID_results.json`, containing the summary written by your reusable evaluation cell;
3. any generated plots, screenshots, or small helper files needed by your notebook.

Your notebook must show:

- Part A hand traces and formulation answers;
- Part B graph search comparison tables and plots;
- Part C heuristic analysis;
- Part D plan diagnosis and forward planner results;
- the reusable evaluation cell for assignment-schema JSON files.

Your submitted code must run with Python 3.11 or later using only the Python standard library and any course-provided `ai9414` helpers. If you use `ai9414`, you remain responsible for understanding and explaining your own solver code.

11 Deadline and questions

Deadline: Friday, 3 July 2026, 5:00 PM AEST.

Late submissions: a penalty of 5% of the maximum mark per day applies to late submissions. If you submit more than 5 days late, you will receive a mark of zero.

Please use the Moodle forum for assignment questions. We will prioritise questions asked in the forum. Do not share code or complete answers publicly. If necessary, contact course staff by email. For discussion-session questions, contact your tutor directly.

12 Academic integrity

You must complete this assignment individually. You may discuss concepts and high-level approaches with other students, consult course materials, consult course staff, and use AI tools for learning concepts or debugging. You must not share code, copy code, submit code written by someone else, or post assignment content or solutions on external forums.

You are responsible for understanding every part of your submission. During the discussion session, your tutor may ask you to explain design choices, trace a frontier update, modify a small case, or diagnose why a planning action is not applicable.

A Suggested ai9414.strips planning helpers

For Part D, you may use the following helper functions from `ai9414.strips`. These helpers remove low-level STRIPS bookkeeping, but your code must still implement the forward BFS loop, visited-state handling, goal checking, and assignment metrics.

```
from ai9414.strips import (
    apply_action_signature,
    get_applicable_actions,
    get_initial_facts,
)
```

Table 5: Suggested planning helper functions.

Helper	Meaning
<code>get_initial_facts(problem)</code>	Returns the initial symbolic state as a list of fact tuples.
<code>get_applicable_actions(problem, facts)</code>	Returns the grounded action strings that are applicable in the current state.
<code>apply_action_signature(problem, facts, action)</code>	Applies one grounded action string to the current state and returns the successor fact list.

A typical forward-BFS structure is:

```
start = get_initial_facts(problem)
frontier = [(start, [])]
visited = {state_id(start)}

while frontier:
    facts, plan = frontier.pop(0)
```

```

if goal_satisfied(facts, problem["goal"]):
    return assignment_result(...)

for action in get_applicable_actions(problem, facts):
    next_facts = apply_action_signature(problem, facts, action)
    ...

```

The helper functions work with the supplied office-delivery planning files and with the live `ai9414 demo strips` payload. The assignment function is still named `solve_planning(problem)` and must return the assignment fields in Table 2.

B References

1. Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson.
2. COMP9414 week 1 lecture: Agents and search.
3. COMP9414 week 2 lecture: Planning.
4. The `ai9414` Python package and course demos for graph search and STRIPS planning.
5. Optional further planning reading: Dana Nau, Malik Ghallab, and Paolo Traverso. *Automated Planning: Theory and Practice*. Morgan Kaufmann. This book goes beyond what is required here, but is useful for you if you want a deeper planning treatment.